

Nenadzirano učenje – grupiranje podataka



Sadržaj

- Grupiranje podataka
- k-means clustering algoritam
 - k-means u scikit-learn
- Primjer kompresije slike koristeći k-means

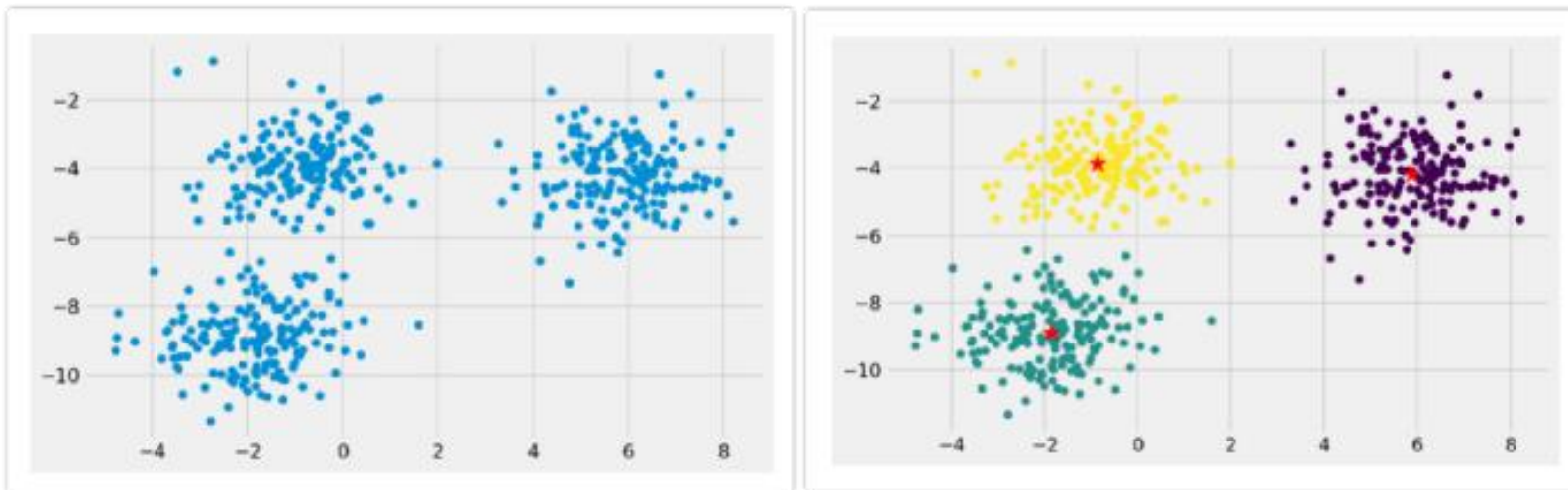
Grupiranje podataka

Grupiranje podataka

- Grupiranje podataka (*clustering*) ili klaster analiza jedan je najčešćih problema nenadziranog strojnog učenja
- **Koristi se kako bi se pronašle grupe ili skrivene zakonitosti i obrasci u podacima;** pokušava se naučiti optimalna podjela podataka
- Podaci su **neoznačeni**; jednom kada se pronađu grupe u podacima, moguće je nove mjerne uzorke dodijeliti odgovarajućoj grupi
- Primjene su raznolike:
 - segmentacija korisničkog ponašanja (npr. prema povijesti kupovine, aktivnosti u aplikaciji i sl.),
 - *text mining*,
 - obrada medicinskih i *automotiv* slika,
 - segmentacija slika,
 - sustavi preporuka,...

Grupiranje podataka

- Svaki klaster koji se pojavljuje u tijeku grupiranja definira jednu grupu objekata koji dijele određeni stupanj sličnosti, ali su više različiti od objekata u drugim klasterima
 - Zbog toga se klasteriranje ponekad naziva i **nenadzirana klasifikacija**
 - ***k-means clustering*** je jedan od najpopularnijih algoritama za grupiranje podataka



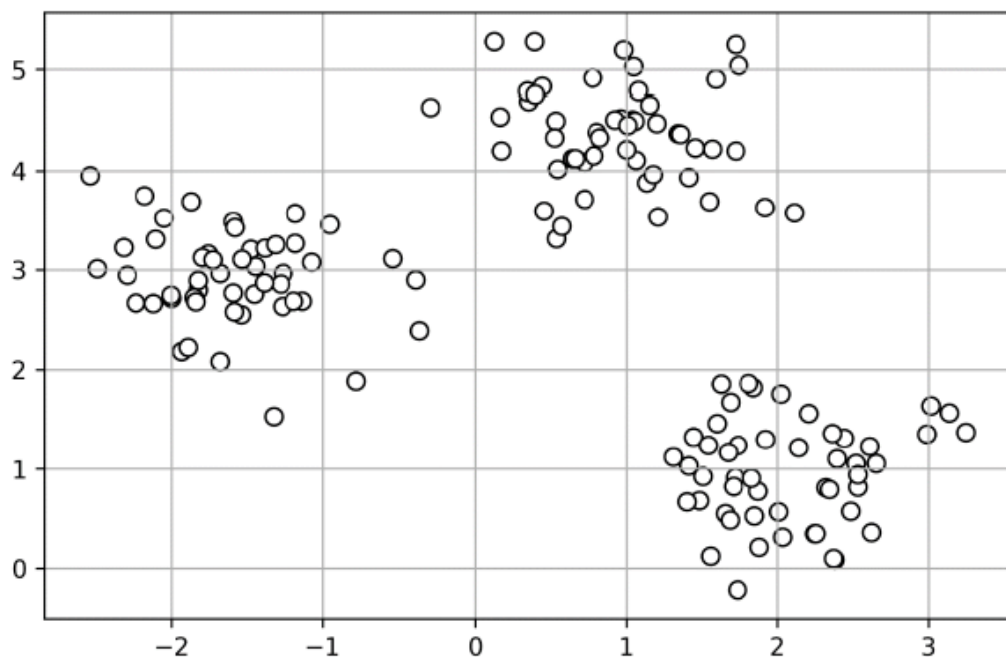
k-means clustering algoritam za
grupiranje podataka

Algoritam „ k srednjih vrijednosti“

- Algoritam k -srednjih vrijednosti
- Osnovni algoritam grupiranja, jednostavan koncept:
 - **Centar (centroid) klastera je aritmetička sredina svih podataka koji pripadaju tom klasteru**
 - **Svaki podatak je bliže svom klasteru nego centrima ostalih klastera**
- Računalno vrlo efikasan
- Iako se klasterizacija metodom K -srednjih vrijednosti može primijeniti na višedimenzionalne podatke, objasniti ćemo princip rada na jednostavnom dvodimenzionalnom skupu podataka radi lakše vizualizacije

Algoritam „ k srednjih vrijednosti“

- **Primjer 1** - Razmotrimo sljedeći skup dvodimenzionalnih podataka
 - 150 generiranih točaka, ulazne veličine tj. atributi svakog podatka (x_1 , x_2)
 - Grubo grupirane u tri regije s većom gustoćom



- Cilj → grupirati primjere na osnovu sličnosti njihovih atributa
- Kako?

Algoritam „ k srednjih vrijednosti“

Koraci k -means clustering algoritma

1. Nasumično odaberite k centroida iz primjera kao inicijalne centre klastera
2. Svaki primjer iz skupa dodijelite najbližem centroidu $\mu^{(j)}$, $j \in \{1, \dots, k\}$
3. Pomjerite centroide do centra primjera koji su im dodijeljeni
4. Ponavljajte korake 2 i 3 dok se dodjele klastera ne promijene ili dok se ne dostigne tolerancija koju je korisnik definirao ili maksimalan broj iteracija
 - Idealno, dok se centroidi ne prestanu pomicati

Algoritam „ k srednjih vrijednosti“

- **Pitanje – kako mjeriti sličnost između objekata (podataka)?**
- Možemo definirati sličnost kao suprotnost rastojanju (udaljenosti)
- Rastojanje za primjere klasterizacije s kontinuiranim (brojčanim) atributima računa se kvadratnim euklidskim rastojanjem (udaljenošću) između dviju točaka $\mathbf{x}$ i $\mathbf{y}$ u m -dimenzionalnom prostoru

$$d(\mathbf{x}, \mathbf{y})^2 = \sum_{j=1}^m (x_j - y_j)^2 = \|\mathbf{x} - \mathbf{y}\|_2^2$$

- Napomena: Indeks j se u ovoj jednadžbi odnosi se j -tu dimenziju (j -ti atribut) ulaza primjera $\mathbf{x}$ i $\mathbf{y}$
- U nastavku ćemo eksponente i i j upotrebljavati za indeks primjera (zapis podatka) i indeks klastera (da ne bi bilo zabune)

Algoritam „k srednjih vrijednosti“

- U nastavku ćemo eksponente i i j upotrebljavati za indeks primjera (zapis podatka) i indeks klastera
- Na osnovu metrike euklidske udaljenosti možemo algoritam k-srednjih vrijednosti opisati kao jednostavan problem optimizacije i kao iterativni pristup za **minimiziranje unutar-klasterskog zbroja kvadratnih grešaka** (engl. *sum of squared errors – SSE*), što se naziva i inercija klastera (eng. *cluster inertia*)

$$SSE = \sum_{i=1}^n \sum_{j=1}^k w^{(i,j)} \|x^{(i)} - \mu^{(j)}\|_2^2 \quad w^{(i,j)} = \begin{cases} 1, & \text{if } x^{(i)} \in j \\ 0, & \text{otherwise} \end{cases}$$

- Ovdje, $\mu^{(j)}$ reprezentativna točka (centroid) za klaster j
- $w^{(i,j)} = 1$ ako se primjer, $x^{(i)}$, nalazi u klasteru j , u suprotnom je 0

Algoritam „k srednjih vrijednosti“

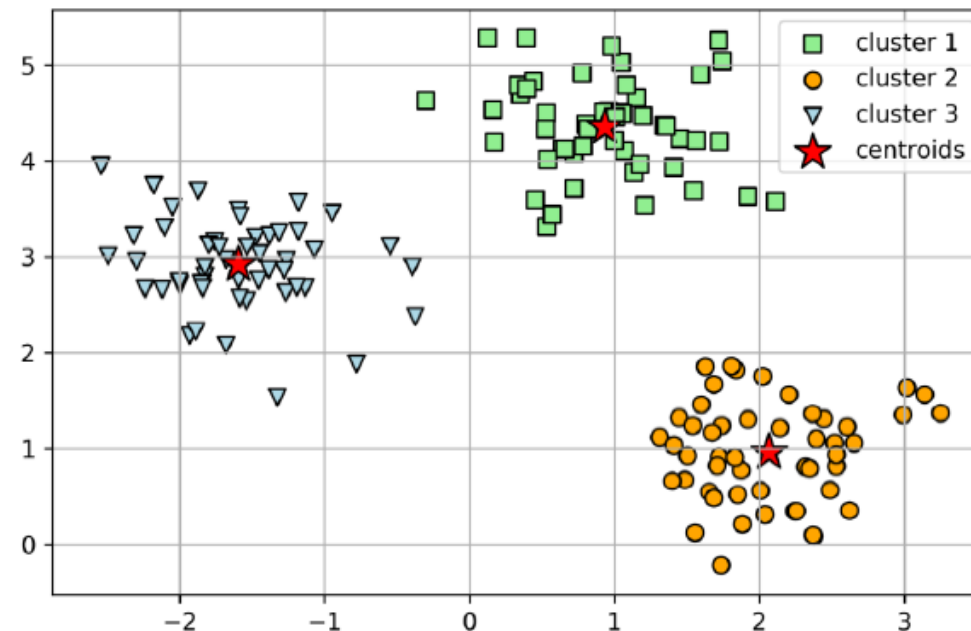
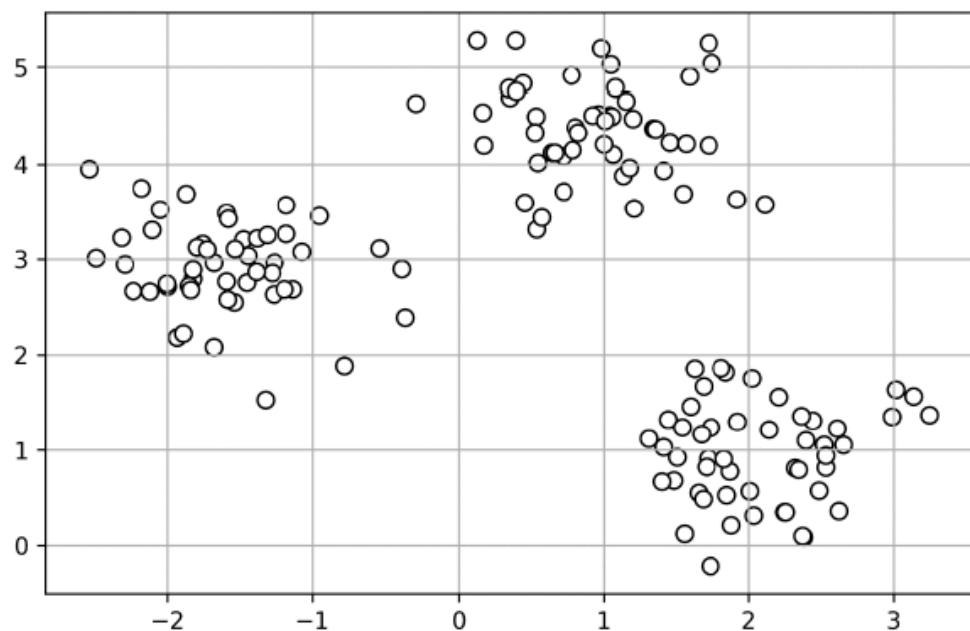
- Kada primjenjujemo metodu k-srednjih vrijednosti na podatke iz realnog svijeta upotrebom metrike euklidske udaljenosti, **moramo biti sigurni da su atributi mjereni na istoj skali** → skaliranje atributa
 - Standardizacija
 - Min-max skaliranje

$$SSE = \sum_{i=1}^n \sum_{j=1}^k w^{(i,j)} \|x^{(i)} - \mu^{(j)}\|_2^2 \quad w^{(i,j)} = \begin{cases} 1, & \text{if } x^{(i)} \in j \\ 0, & \text{otherwise} \end{cases}$$

- Ovdje, $\mu^{(j)}$ reprezentativna točka (centroid) za klaster j
- $w^{(i,j)} = 1$ ako se primjer, $x^{(i)}$, nalazi u klasteru j , u suprotnom je 0

Algoritam „k srednjih vrijednosti“

- **Primjer 1** - Razmotrimo sljedeći skup dvodimenzionalnih podataka
 - 150 generiranih točaka, ulazne veličine tj. atributi svakog podatka (x_1 , x_2)
 - Primjena algoritma k-srednjih vrijednosti

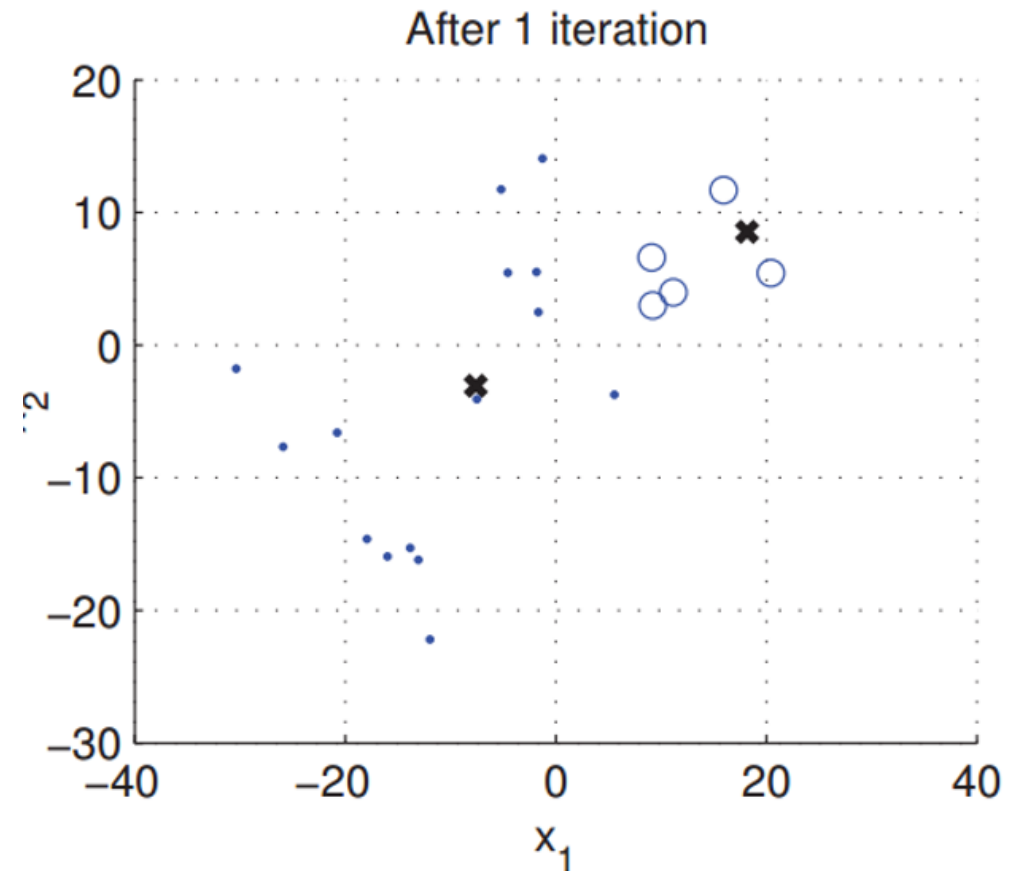
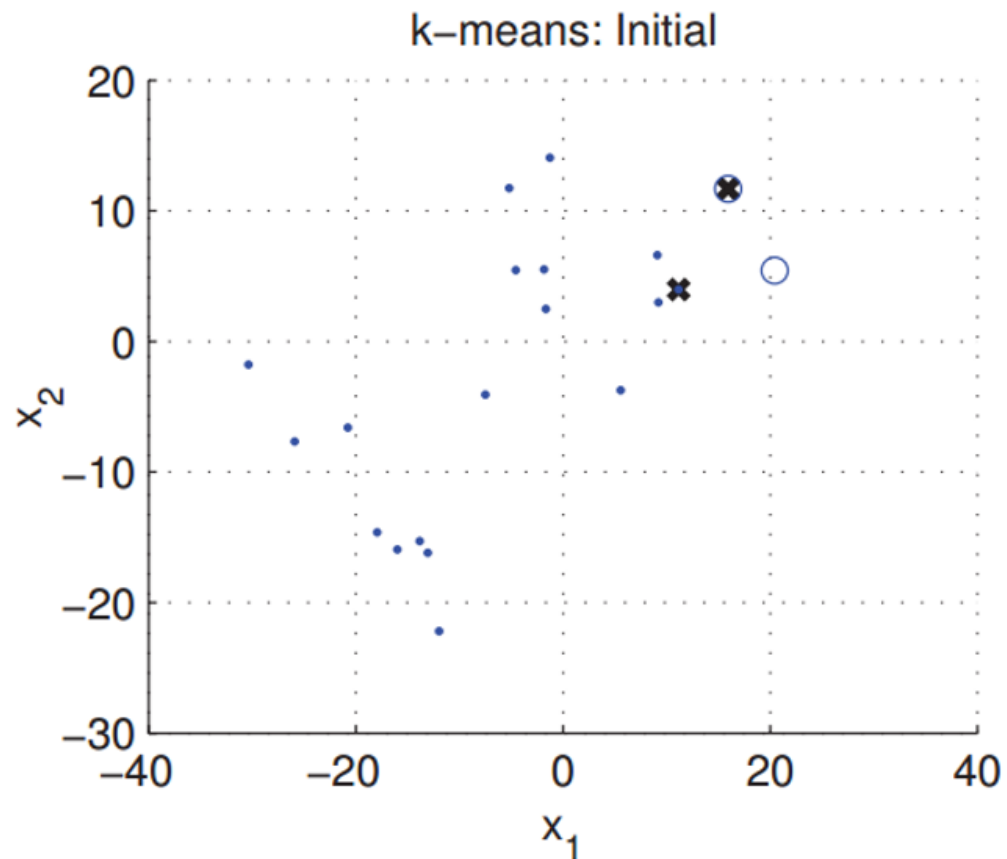


Algoritam „ k srednjih vrijednosti“

- **Primjer 2** - Razmotrimo primjer primjene algoritma k -srednjih vrijednosti na sličnom primjeru...
 - Imamo određene trening podatke na raspolaganju koji su karakterizirani dvama atributima (x_1 i x_2)
- Na početku trebamo definirati formu modela
 - Koliko klasa očekujemo ? $\rightarrow$ npr. 2
 - **To je jedina pretpostavka (moramo dati kao informaciju algoritmu)**
 - Ne znamo pri treningu kojoj klasi pripada koji podatak (**nenadzirano učenje**)

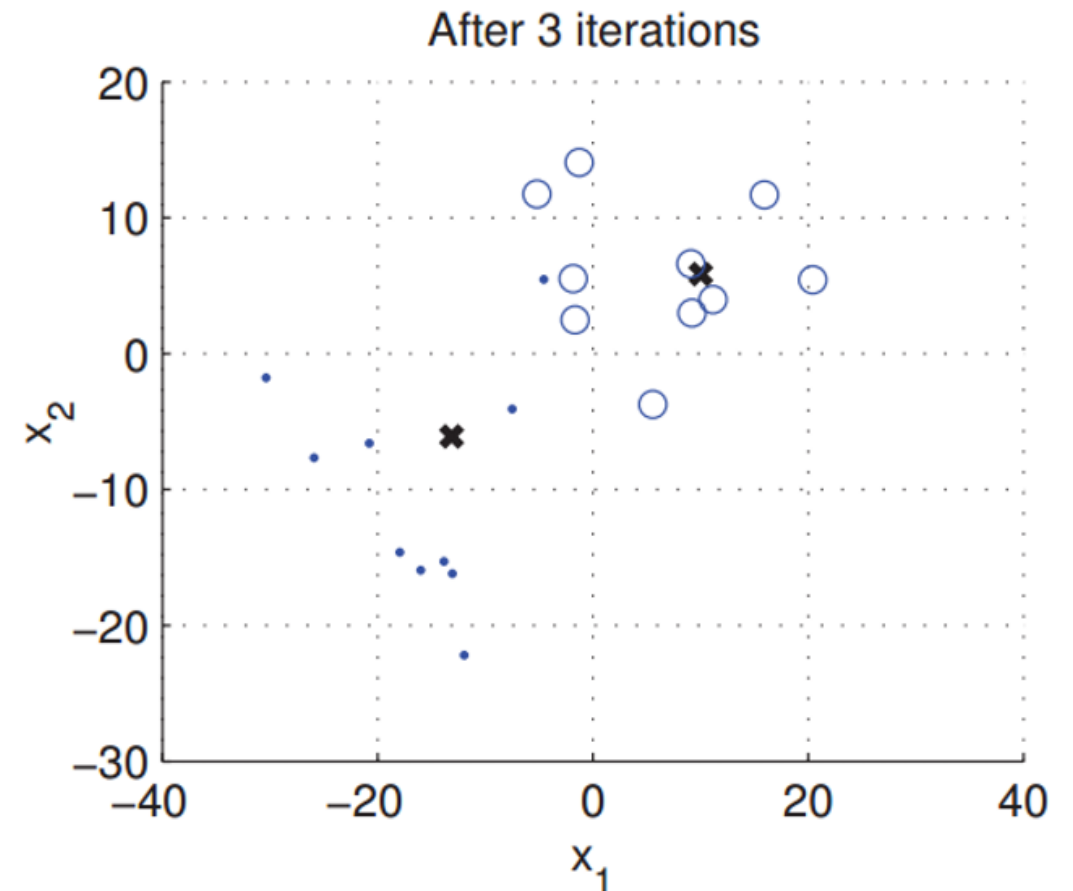
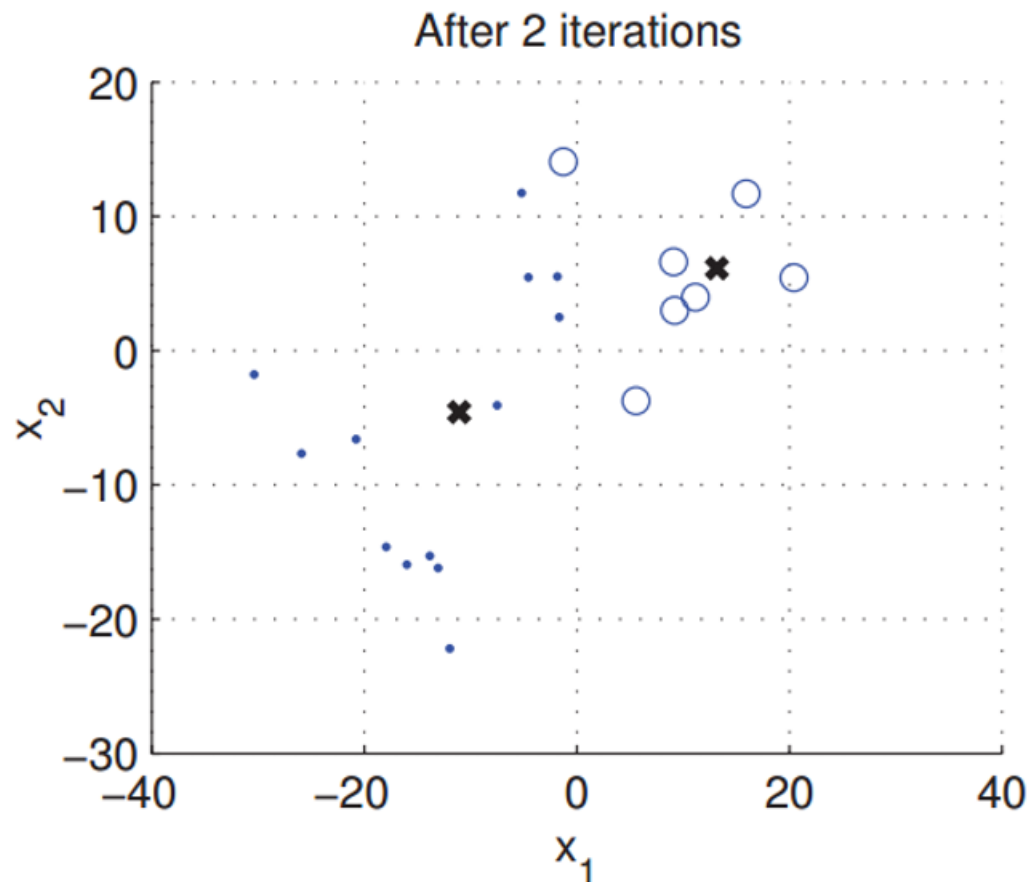
Algoritam „ k srednjih vrijednosti“

- **Primjer uz $k=2$**



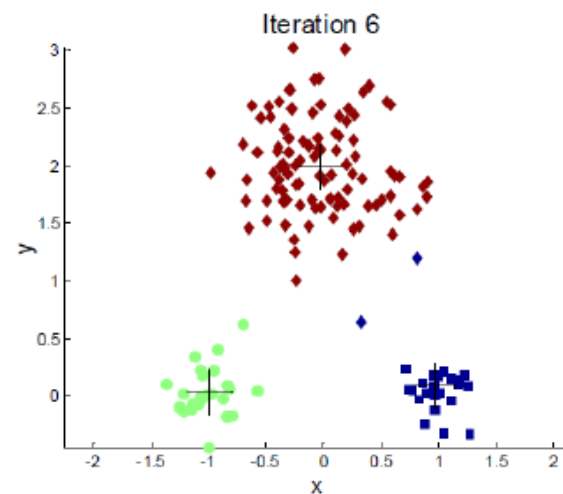
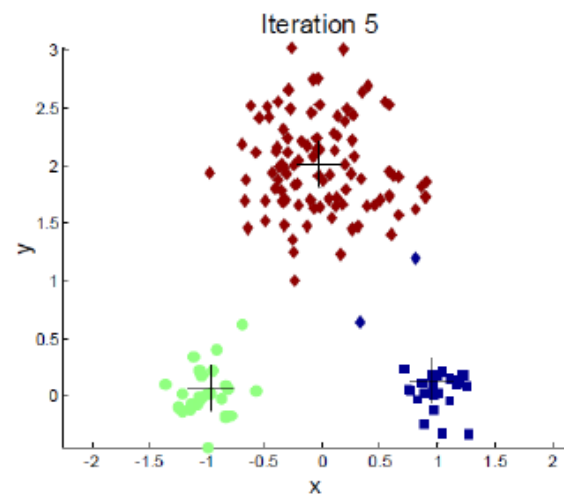
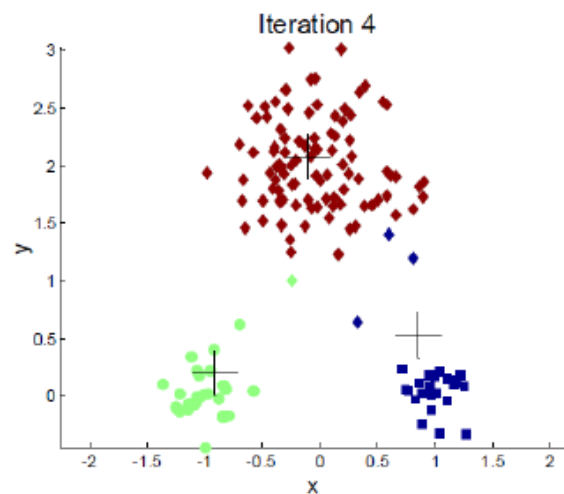
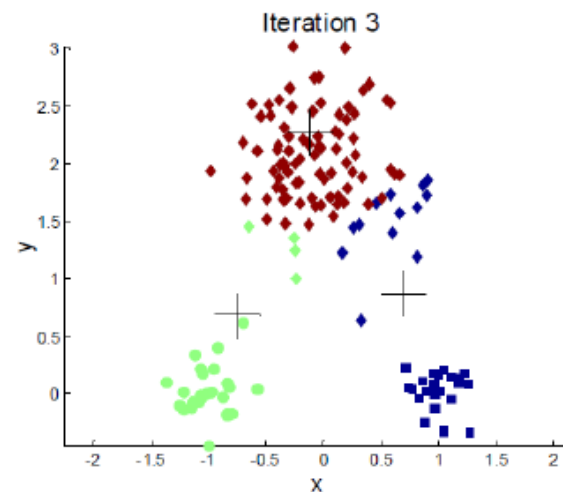
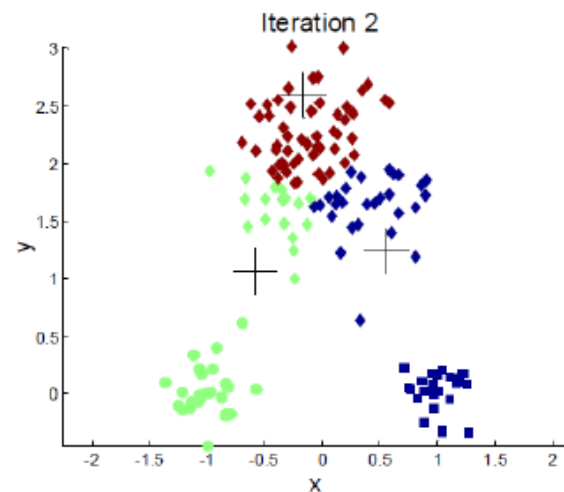
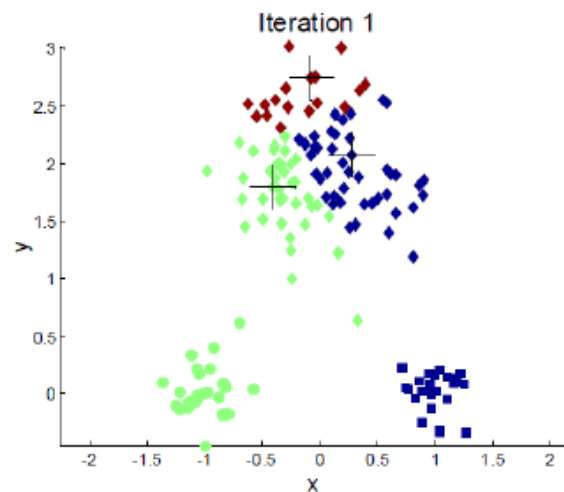
Algoritam „ k srednjih vrijednosti“

- **Primjer uz $k=2$**



Algoritam „k srednjih vrijednosti“

- Primjer:
imamo
određene
treening
podatke na
raspolaganju
koji su
karakterizirani
s 2 vrijednosti
(x_1 i x_2), ali
očekujemo $k=3$
klase podataka

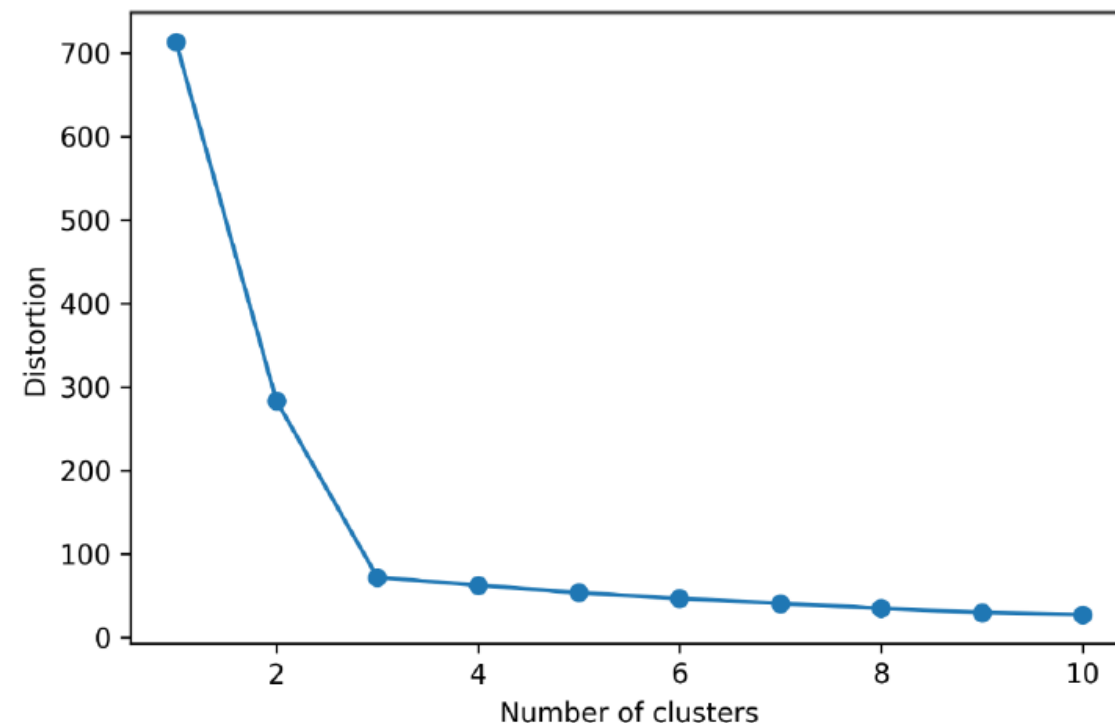


Kako odabrati broj klastera?

- Jedan od glavnih izazova u nenadziranom učenju je to što ne znamo konačan odgovor
- Nemamo istinite oznake klasa (labele) koje nam omogućuju primjenu tehnika koje smo koristili ranije kod nadziranog učenja
- Kao i kod svakog algoritma učenja, potrebno je podesiti odgovarajuće parametre koji kontroliraju složenost modela
 - U slučaju algoritma k-srednjih vrijednosti, to je broj klastera k
- Da bismo kvantificirali kvalitetu grupiranja u klastere, moramo upotrijebiti primarnu metriku, kao što je SSE (**često se naziva i distorzija**) unutar klaster, da bismo usporedili izvedbu različitih klasteriranja algoritmom k-srednjih vrijednosti

Kako odabrati broj klastera?

- Na osnovu SSE unutar klastera, možemo koristiti tzv. **metodu lakta** (engl. *elbow method*), za procjenu optimalnog broja klastera, k , za dani zadatak
- Možemo reći da ako k raste, SSE će se smanjiti zato što će primjeri biti bliži centroidima kojima su dodijeljeni
- Ideja metode lakta je identificirati vrijednost k u kojoj SSE počinje najbrže rasti, što će postati jasnije ako nacrtamo SSE u za različite vrijednosti k



→ Ovdje se lakat nalazi u $k = 3$, što ukazuje na to da je $k=3$ dobar izbor za ovaj skup podataka

k-means clustering algoritam u scikit-learn

- K-means algoritam je implementiran unutar klase *KMeans* iz *cluster* modula scikit-learn biblioteke

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++', n_init=10, max_iter=300, tol=0.0001, verbose=0, random_state=None, copy_x=True, algorithm='lloyd')
```

- *n_clusters* – broj klastera – moramo ga *a priori* postaviti (ograničenje k-means-a)
- *init* – način odabira početnih (inicijanih) centroida
 - 'random' – nasumičnim odabirom odabire k podatkovnih primjera kao početne centroide
 - 'kmeans++' – „pametniji” način postavljanja inicijalnih centroida → da budu daleko jedan od drugog → dovodi do boljih i konzistentnijih rezultata (obično u manjem broju iteracija)

k-means clustering algoritam u scikit-learn

- K-means algoritam je implementiran unutar klase *KMeans* iz *cluster* modula scikit-learn biblioteke

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++', n_init=10, max_iter=300, tol=0.0001, verbose=0, random_state=None, copy_x=True, algorithm='lloyd')
```

- *n_init* – broj nezavisnih pokretanja k-means algoritma → kao finalni model bit će odabran onaj model koji daje najmanji SSE, tj. najmanju *inertia* (ima više smisla za *init='random'*)
- *max_iter* – maksimalni broj iteracija za svako pojedinačno pokretanje algoritma
 - Zaustavlja se ranije ako se konvergira prije nego što je dostignut maksimalni broj iteracija
 - Moguće da ne dostigne konvergenciju za određeno pokretanje pa je potreban velik broj iteracija → računalno skupo

k-means clustering algoritam u scikit-learn

- K-means algoritam je implementiran unutar klase *KMeans* iz *cluster* modula scikit-learn biblioteke

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++', n_init=10, max_iter=300, tol=0.0001, verbose=0, random_state=None, copy_x=True, algorithm='lloyd')
```

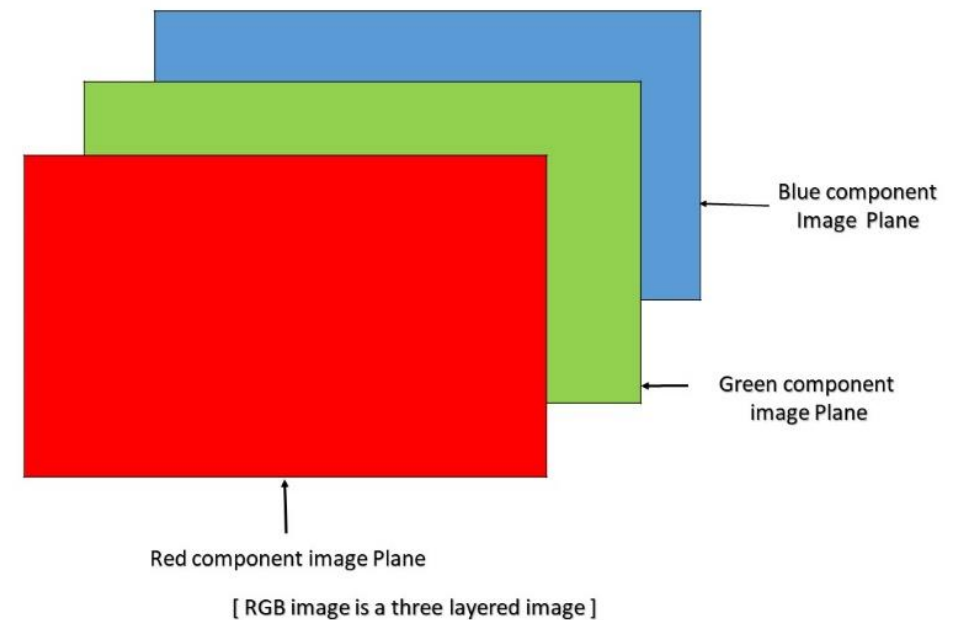
- *tol* – kontrolira toleranciju u odnosu na promjene u SSE-u unutar klastera za deklariranje konvergencije
- ...

k-means clustering algoritam u scikit-learn

- Najvažniji **atributi** su:
 - **cluster_centers** – koordinate centara klastera
 - **labels** – labele svake točke
 - **inertia** – suma kvadratnih odstupanja svih uzoraka od njima najbližeg centra klastera
- Najvažnije **metode** su:
 - **fit(X[, y, sample_weight])** – proračunava k-means klasteriranje
 - **predict(X, sample_weight=None)** – za svaki podatak iz X predviđa kojem klasteru pripada
- Detaljnije pojašnjenje za sve parametre i attribute može se pronaći ovdje [KMeans](#)

Primjer kompresije slike koristeći *k-means*

- Podsjetimo se da za 8-bitnu RGB sliku u boji
 - svaki piksel je predstavljen s tri boje (crvena, zelena i plava)
 - svaka boja je predstavljena 8-bitnim cijelim brojem – svaki kanal ima nijanse od 0 do 255
 - svaki piksel je predstavljen s tri bajta
 - ukupno ima $256 \times 256 \times 256 = 16.777.216$ mogućih kombinacija ili boja
- Koliko boja može vidjeti čovjek? Moguće nekoliko milijuna, ovisi o pojedincu i zaslonu koji koristimo



Primjer kompresije slike koristeći *k-means*

- Ideja je grupirati slične boje u k klastera (npr. 16) različitih boja (RGB vrijednosti)
- Svaki centroid klastera je predstavnik vektora boja u RGB prostoru boja svog odgovarajućeg klastera
- Ovih k centroida klastera zamijenit će sve vektore boja u svojim klasterima
- Dakle, trebamo samo pohraniti oznaku (label) za svaki piksel koja govori kojem klasteru taj piksel pripada
 - U slučaju $k=16$ trebamo samo 4 bita po pikselu (umjesto 24 bita u izvornoj RGB slici)
 - I vektor kodnih riječi (engl. *codebook*) koji nam govori koja je boja određene oznake

Primjer kompresije slike koristeći *k-means*

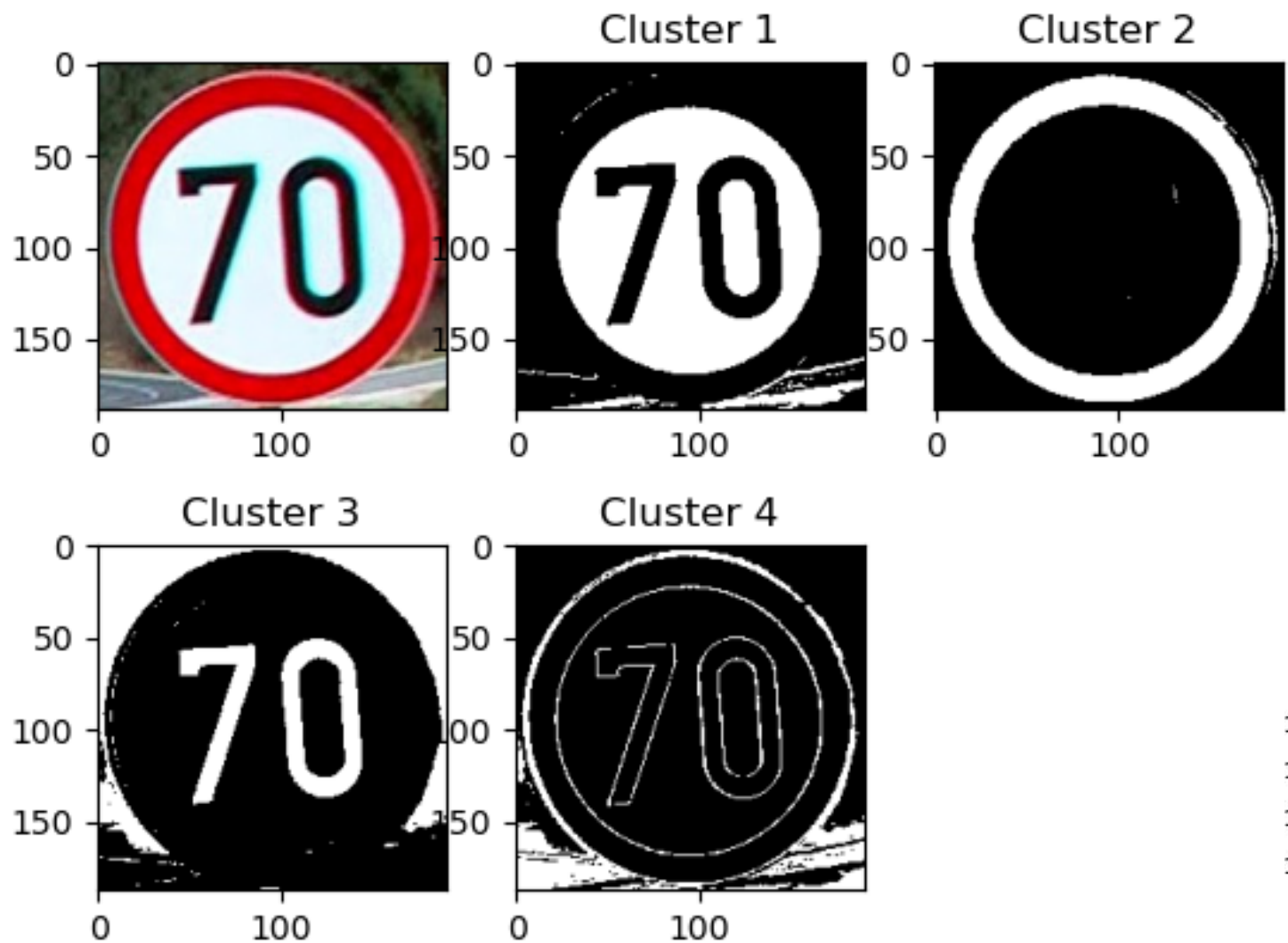
30201 boja



16 boja

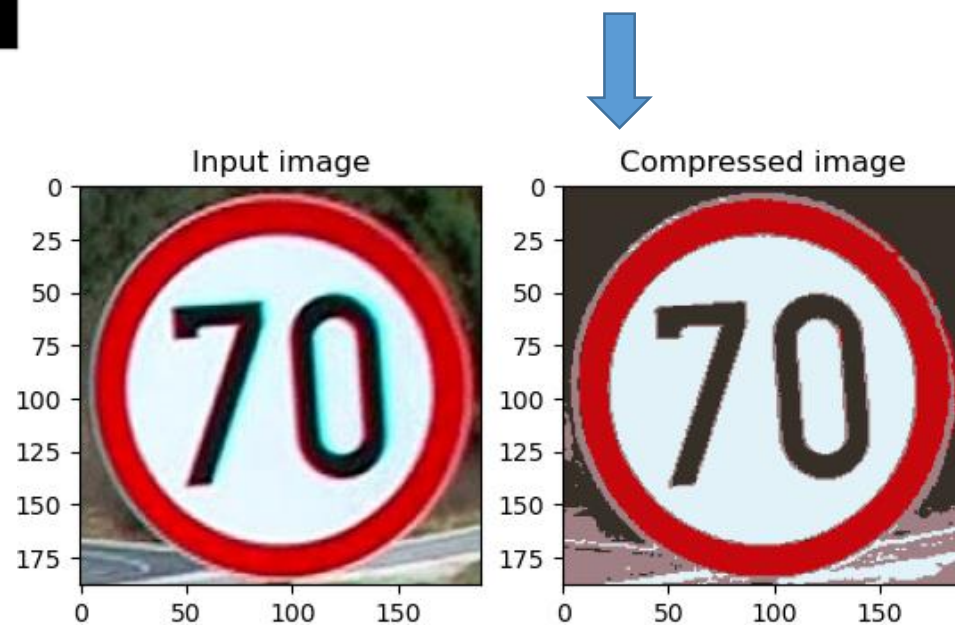


Primjer kompresije slike koristeći *k-means*



← Originalna slika i 4 klastera

Originalna slika i slika nakon k-means uz $k=4$



P i t a n j a ?